Сравнение реализации GDRM

Веб-модуль проекта RGSCenter и desktop-приложение GDRMonitor

RGSCenter	GDRMonitor desktop	Главный вывод
Серверный модуль для автоматического подключения, фонового опроса GDRM, публикации текущего состояния и удалённого просмотра через браузер.	Инженерная Swing-программа для локального мониторинга, графиков, ручного подключения и настройки параметров прибора.	RGSCenter переносит основное чтение измерений и делает транспорт безопаснее, но пока не заменяет сервисные функции desktop-программы.

Критерий	RGSCenter	GDRMonitor desktop
Основной сценарий	Автоматическая работа 24/7	Локальная инженерная работа
Мониторинг GDRM	Основные измерения реализованы	Полный мониторинг и графики
Сервисные команды	Минимальный набор	Развитое чтение и изменение параметров
Потоки и очереди	Безопасные ограниченные очереди	Наследуемая многопоточная схема с рисками
Удалённый доступ	Да, через браузер и Spring Security	Нет, локальное desktop-приложение

Объект сравнения

Сравниваются текущая серверная интеграция прибора GDRM в RGSCenter и исходная desktop-реализация GDRMonitor. Оценка относится к существующему коду: это не сравнение языков или UI-фреймворков, а анализ фактических функций, транспорта, потоков, хранения данных и эксплуатационных сценариев.

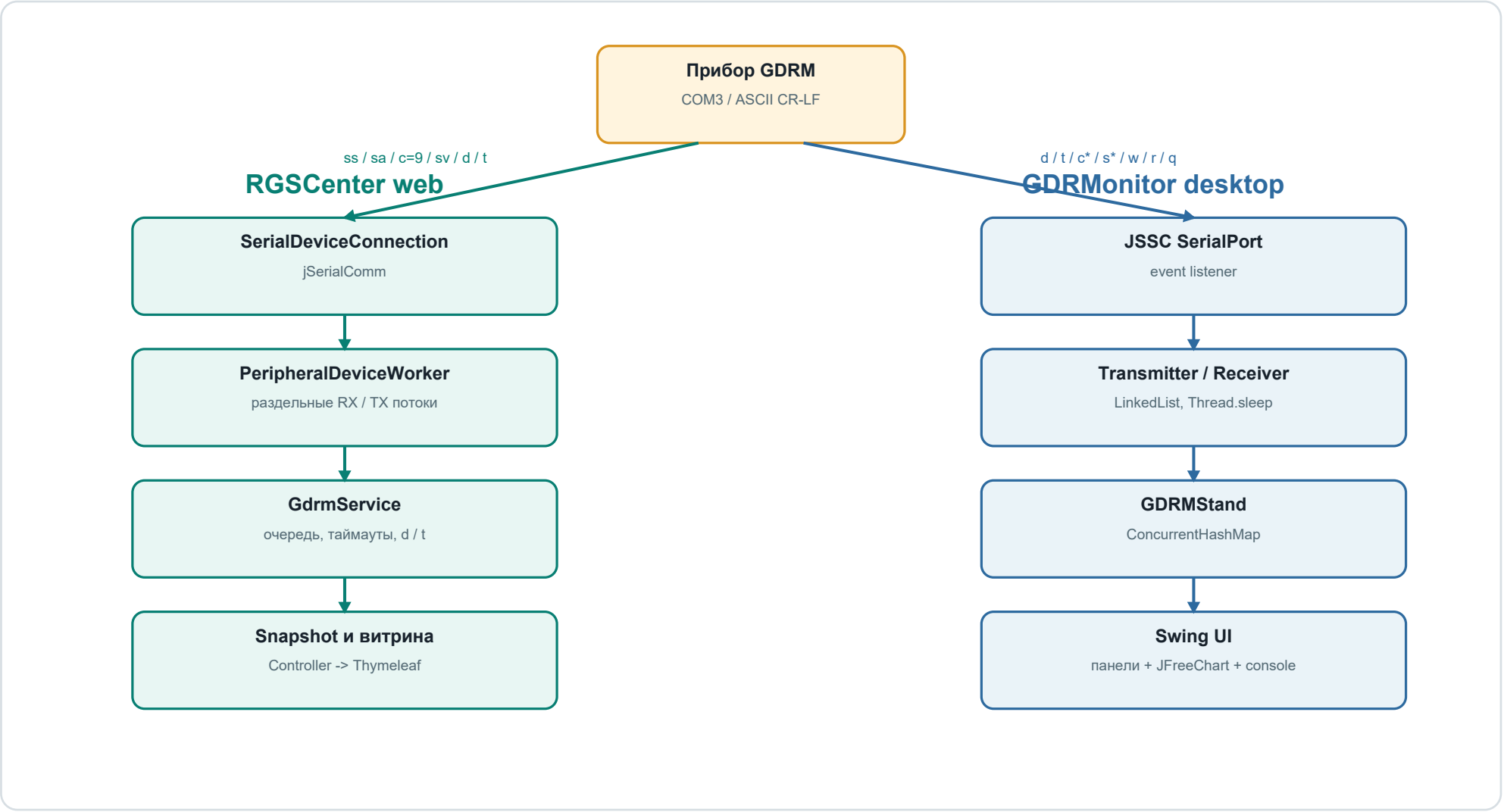
1. Функциональное сравнение

Область	RGSCenter web	GDRMonitor desktop	Вывод
Назначение	Фоновый опрос и публикация состояния через web	Локальный монитор и сервисная программа	Разные роли
Интерфейс	Thymeleaf: /gdrm и /gdrm/status	Java Swing	Web удобнее удалённо
COM-библиотека	jSerialComm	JSSC	Обе применимы
Выбор COM	Конфигурация, по умолчанию COM3	Выбор оператором в UI	Разные сценарии
Подключение	Автоматически при запуске	Кнопка Connect	RGSCenter для службы
Скорость	Автопоиск 9600/115200	Ручное управление и смена	RGSCenter автономнее
Периодический опрос	@Scheduled, по умолчанию 1 с	Собственные циклы и потоки	RGSCenter проще сопровождать
Команды чтения	ss, sa, c=9, sv, d, t	d, t и расширенное чтение параметров	Базовый паритет
Команды записи	Не опубликованы в UI	c*, w, r, q, console	Desktop функциональнее
Измерения	4 канала, среднее, ошибка, доза, время	Те же данные	Высокое соответствие
Operating data	HV, температура, влажность, давление, статус	Те же и сервисные параметры	Частичное соответствие
Графики	Нет	JFreeChart, временные ряды	Пробел web

2. Архитектура, эксплуатация и безопасность

Область	RGSCenter web	GDRMonitor desktop	Вывод
История	Только текущий snapshot	Графическая история сеанса	Нужно развивать
Инженерная консоль	Нет	Есть	Нужна только сервисной роли
Хранилище	AtomicReference snapshot + CurrentDataService	GDRMStand / ConcurrentHashMap	Web лучше отделяет UI
Очередь команд	Bounded blocking queue	Обычный LinkedList	RGSCenter безопаснее
Буфер	ByteRingBuffer + ReentrantLock	Самописный CircleBuffer	RGSCenter безопаснее
Ожидание	Блокирующее, с таймаутами	Thread.sleep и активные циклы	RGSCenter эффективнее
Остановка	Interrupt и контролируемый lifecycle	Встречается Thread.stop	RGSCenter безопаснее
Отказы	Таймауты, диагностика, переидентификация	Преимущественно локальное состояние UI	RGSCenter устойчивее
Доступ	Браузер, несколько пользователей	Один локальный пользователь	Преимущество web
Безопасность	Spring Security	Без web-аутентификации	Преимущество web
Тесты	Protocol/ViewModel tests	Формальные unit-тесты не обнаружены	Преимущество web
Next.js	GDRM ещё остаётся на Thymeleaf	Не применимо	Следующий web-инкремент

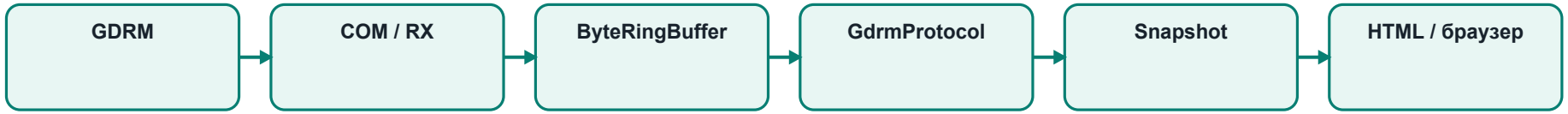
3. Сравнительная архитектурная схема



В RGSCenter web-запрос читает уже подготовленный snapshot и не обращается к COM-порту напрямую. В GDRMonitor пользовательский интерфейс теснее связан с транспортом и одновременно является средством настройки прибора.

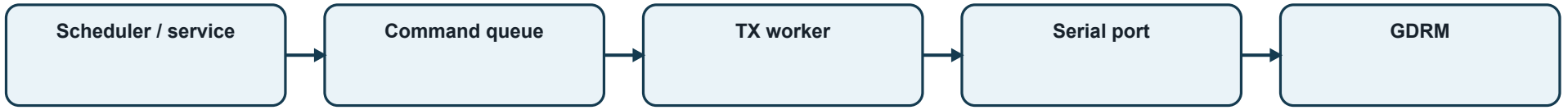
4. Потоки данных и команд

Чтение данных в RGSCenter



Данные проходят последовательный разбор и публикуются как неизменяемое текущее состояние

Отправка команд



Очередь ограничена; worker ожидает команду без активного опроса и передаёт её с контролем lifecycle

Что изменено относительно desktop-схемы



5. Выводы и рекомендуемая последовательность развития

Направление	Оценка	Обоснование
Соответствие мониторинга	**Высокое**	Основные измерения формата 9, четыре канала, средние значения и operating data уже отображаются в RGSCenter.
Соответствие сервисных функций	**Низкое**	В web пока нет редактора параметров, графиков, консоли, reset/save и управляемой записи настроек.
Надёжность транспорта	**Выше в RGSCenter**	Очереди ограничены, буфер защищён, ожидания блокирующие, завершение потоков контролируемое.
Готовность к эксплуатации 24/7	**Выше в RGSCenter**	Автоподключение, таймауты, переидентификация, логирование, security и отделение UI от COM.
Готовность к сервисным работам	**Выше в GDRMonitor**	Desktop остаётся более полным инженерным инструментом диагностики и настройки прибора.

Рекомендуемые инкременты

1. Добавить GET /api/v1/gdrm и стабильные DTO текущего состояния для Next.js.
2. Создать Next.js-страницу /gdrm, сохранив текущую Thymeleaf-страницу на время миграции.
3. Добавить серверную временную историю и графики мощности дозы и статистической ошибки.
4. Разделить права: оператор - просмотр; сервисный инженер - изменение параметров; администратор - подключение и учётные записи.
5. Реализовать только необходимые команды записи с валидацией, таймаутом, подтверждением результата и audit log.
6. Raw-console оставить отдельным сервисным режимом, отключённым в production по умолчанию.

Итоговая позиция

RGSCenter следует развивать как основной серверный слой управления комплексом, а GDRMonitor использовать как источник протокольной логики и эталон инженерных функций. Переносить устаревшую модель потоков desktop-приложения в RGSCenter не требуется.